

Instituto Tecnológico de San Juan del Río



Tópicos de Ciberseguridad

R002

P R E S E N T A :

Oscar Gabriel Rogelio Jiménez Hernández 22590332

Roberto Joshua García Solís 22590410

Ingeniería en Sistemas Computacionales

PROFESOR: Leonardo Valdez Arteaga

PERIODO[Agosto-Diciembre2026]

1. Objetivo

Evadir la validación de clave (serial key) del ejecutable T.exe mediante ingeniería inversa, modificando el salto condicional que decide el acceso, de modo que la aplicación conceda el acceso con cualquier clave.

2. Herramientas utilizadas

- **Sistema operativo:** Linux (CachyOS, base Arch).
- **radare2** (o Cutter) para desensamblar, localizar y parchar la instrucción.
- **Python 3** para automatizar el parche a partir del binario original.

3. Identificación del binario

```
Archivo:T.exe
Tipo      :PE32executable(console)Intel180386,MSWindowsTamaño
          : 2048 bytes
SHA256    :74c399e18e34ffd0fe04718edaac237e254736da671b4b64a69ec519d12192fd
```

4. Rastreo de cadenas (Strings)

Al inspeccionar las cadenas del binario se identifica la clave y los mensajes de la validación:

- Clave esperada: NohackPWX (en 0x402000)
- Mensaje de éxito: "Acceso Concedido" (0x402027)
- Mensaje de error: "Acceso no Concedido" (0x402039)

El programa usa `strcmp` para comparar el argumento contra NohackPWX. (De hecho, al estar la clave visible, bastaría introducirla; el objetivo del ejercicio es evadir la comprobación por medio del parche.)

```
~/Documentos/AGODIC2026_TOPICOS/Tema1/Negro/R002 main*
) rabin2 -z T.exe | grep -Ei 'nohack|acceso|contrase'
0  0x00000060 0x00402000 9  10  .data  ascii NohackPWX
1  0x0000006a 0x0040200a 27 29  .data  utf8  No colocaste la contraseña\n
2  0x000000627 0x00402027 17 18  .data  ascii Acceso Concedido\n
3  0x000000639 0x00402039 20 21  .data  ascii Acceso no Concedido\n
```

5. Localización de la instrucción en el depurador

Rastreando desde las cadenas se llega a la rutina de validación. El bloque relevante es:

Dirección	Bytes	Instrucción
00401045	83 F8 00	cmp eax, 0 ; ¿strcmp == 0?
00401048	0F 85 13 00 00 00	jne 0x401061 ← salto a neutralizar
0040104E	B8 27 20 40 00	mov eax, "Acceso Concedido"
00401061	B8 39 20 40 00	mov eax, "Acceso no Concedido"

`cmp eax, 0` evalúa el resultado de `strcmp` (0 = clave correcta). Si es igual, `jne` *no* salta y se ejecuta `printf("Acceso Concedido")`; si *no* es igual, `jne` salta a 0x401061 y niega el acceso. El salto `jne` está en el offset físico 0x248.

```

~/Documentos/AG00IC2026_TOPICOS/Tema1/Negro/R002 main*
) r2 -q -c 's 0x00401048; pd 4' T.exe
WARN: Relocs has not been applied. Please use '-e bin.relocs.apply=true' or '-e bin.cache=true' next time
< 0x00401048 0f8513000000 jne 0x401061
0x0040104e b827204000 mov eax, str.Acceso_Concedido_n ; 0x402027 ; "Acceso Concedido\n"
0x00401053 50 push eax
0x00401054 e8cf010000 call 0x401228

```

6. Aplicación del parche

Para evadir la validación se **elimina el salto condicional**: se reemplaza la instrucción jne (6 bytes) por seis nop, de modo que la ejecución siempre continúe hacia el bloque de acceso concedido:

	Bytes	Significado
Original	0F 85 13 00 00 00	jne 0x401061
Parchado	90 90 90 90 90 90	nop x6

Tras el parche, el desensamblado muestra seis nop en lugar del salto, y la ejecución cae directamente en printf("Acceso Concedido").

```

~/Documentos/AG00IC2026_TOPICOS/Tema1/Negro/R002 main*
) cp T.exe T_parche.exe

~/Documentos/AG00IC2026_TOPICOS/Tema1/Negro/R002 main*
) r2 -w -q -c 's 0x00401048; wx 909090909090; pd 6' T_parche.exe
WARN: Relocs has not been applied. Please use '-e bin.relocs.apply=true' or '-e bin.cache=true' next time
0x00401048 90 nop
0x00401049 90 nop
0x0040104a 90 nop
0x0040104b 90 nop
0x0040104c 90 nop
0x0040104d 90 nop

```

7. Automatización con Python

El script parche_T.py reproduce el parche de forma automática: localiza el offset 0x248, verifica que ahí estén los bytes 0F 85 13 00 00 00 y los sustituye por 90 90 90 90 90 90, generando un archivo nuevo sin modificar el original.

- Verifica que el archivo exista y que los bytes sean los esperados (si no, se detiene).
- Crea un respaldo .bak del original.
- Escribe T_parche.exe con los seis bytes sustituidos.

El código fuente completo se entrega en parche_T.py.

8. Comparación del original contra el parchado

Aplicando una comparación binaria entre T.exe y T_parche.exe se confirma que **solo cambian seis bytes**, en el offset esperado, sin variar el tamaño:

```

Tamañoidéntico: sí
Bytesdistintos: 0x248..0x24D      (0F8513000000      ->  909090909090)

```

```
~/Documentos/AGODIC2026_TOPICOS/Tema1/Negro/R002 main*
) cmp -l T.exe T_parche.exe
585 17 220
586 205 220
587 23 220
588 0 220
589 0 220
590 0 220
```

9. Resultados

1. El binario parchado concede el acceso con cualquier clave (imprime “Acceso Concedido”).
2. El salto condicional que dependía de `strcmp` quedó neutralizado.
3. Se cumple el objetivo: pasar la validación sin conocer/usar la clave correcta.

```
~/Documentos/AGODIC2026_TOPICOS/Tema1/Negro/R002 main*
) wine T.exe
MESA-EGLError: pci id for fd 35: 10de:25a2, driver (null)
pci id for fd 36: 10de:25a2, driver (null)
pci id for fd 37: 10de:25a2, driver (null)
MESA-EGLError: egl: failed to create dri2 screen
MESA-EGLError: pci id for fd 35: 10de:25a2, driver (null)
pci id for fd 36: 10de:25a2, driver (null)
pci id for fd 37: 10de:25a2, driver (null)
MESA-EGLError: egl: failed to create dri2 screen
MESA-EGLError: pci id for fd 35: 10de:25a2, driver (null)
No colocaste la contraseña

~/Documentos/AGODIC2026_TOPICOS/Tema1/Negro/R002 main*
) wine T_parche.exe 12345
MESA-EGLError: pci id for fd 35: 10de:25a2, driver (null)
pci id for fd 36: 10de:25a2, driver (null)
pci id for fd 37: 10de:25a2, driver (null)
MESA-EGLError: egl: failed to create dri2 screen
MESA-EGLError: pci id for fd 35: 10de:25a2, driver (null)
pci id for fd 36: 10de:25a2, driver (null)
pci id for fd 37: 10de:25a2, driver (null)
MESA-EGLError: egl: failed to create dri2 screen
MESA-EGLError: pci id for fd 35: 10de:25a2, driver (null)
Acceso Concedido
```